UNIVERSITI TEKNOLOGI MARA
GROUP PROJECT

COURSE	:	**AUTOMATION AND ROBOTIC**
COURSE CODE	:	**ESE689**
SEMESTER	:	**8**

"I hereby declare that all submission is based on my original efforts, not plagiarised from any source(s), nor attempting to or communicating with any other person. I am fully aware that candidates who are suspected of violating this pledge are liable to be referred to the Disciplinary Committee of the Faculty."

NAME : **ANIS SUHANA BINTI MOHD AZMI**

DAHLIA FATEHA BINTI AZIZI

UiTM NO. : **2023816852**

2023262198

GROUP : **CEEE222T8A**

LECTURER : **DR RUHIZAN LIZA BINTI AHMAD SHAURI**

Task	CO	PO	Level of Difficulty	WP, WKs, SDG	MARKS EARNED
1	CO3	PO3	C4	WP2	/10
2	CO3	PO3	C2	WP1	/5
3	CO3	PO3	C3	WP2	/10
4	CO3	PO3	C6	WP3	/15
				MARKS	/40

TABLE OF CONTENT

1.0 INTRODUCTION 1

2.0 PROBLEM ANALYSIS AND CONSEQUENCES 2

3.0 AUTONOMOUS NAVIGATION METHOD 4

3.1 Robot Operating System (ROS) 4

3.2 LiDAR Sensor 4

3.3 Simultaneous Localization and Mapping (SLAM) 4

3.4 Dynamic Window Approach (DWA) Local Planner 5

3.5 Robot Visualization (RViz) 5

4.0 AUTONOMOUS NAVIGATION STEP 6

5.0 EXPERIMENTAL RESULTS AND DISCUSSION 11

6.0 SAFETY AND ENVIRONMENTAL CONSIDERATION 13

7.0 CONCLUSION 14

8.0 REFERENCES 15

1.0 INTRODUCTION

Autonomous mobile robots are becoming increasingly important in various fields such as industrial automation, warehouse management, logistics and service robotics because they are capable of performing navigation tasks with minimal human intervention. By integrating technologies such as the Robot Operating System (ROS), Light Detection and Ranging (LiDAR), Simultaneous Localization and Mapping (SLAM) and navigation planners, a robot is able to identify its surroundings, estimate its current position and move safely towards a specified destination. However, developing a reliable autonomous navigation system is still a challenging task because several processes, including communication between the computer and the robot, map generation, localization, path planning and obstacle detection, must work together correctly. If any of these processes fail, the robot may generate an incorrect path, collide with surrounding obstacles or fail to reach its intended destination. Therefore, a reliable navigation system is required to ensure that the robot can operate safely, accurately and efficiently. In this project, the Bveeta Mini robot is used to develop and evaluate an autonomous navigation system using ROS together with a LiDAR sensor, Hector SLAM and the Dynamic Window Approach (DWA) Local Planner. The project begins by establishing communication between the computer and the robot, followed by generating and saving a map of the surrounding environment, performing localization and implementing autonomous navigation. Finally, a Python-based sequential goal navigation program is used to allow the robot to move autonomously to three predefined destinations in sequence. Therefore, the objectives of this project are to configure communication between the computer and the Bveeta Mini robot, implement an autonomous navigation system using LiDAR mapping and the ROS navigation stack, and verify the robot's ability to navigate safely and accurately to multiple destinations through simulation and experimental testing.

2.0 PROBLEM ANALYSIS AND CONSEQUENCES

In this project, the Bveeta Mini robot needs to move autonomously to three coordinates that have been set using the Robot Operating System (ROS). To ensure smooth navigation, several processes such as communication between the computer and the robot, map construction, localization, path planning and obstacle detection need to work properly. If any of these processes fail as shown in Figure 2.1, the robot's navigation performance will be affected and the robot may not be able to reach the desired destination. Therefore, this section discusses the main problems that can occur and their consequences for the autonomous navigation system.

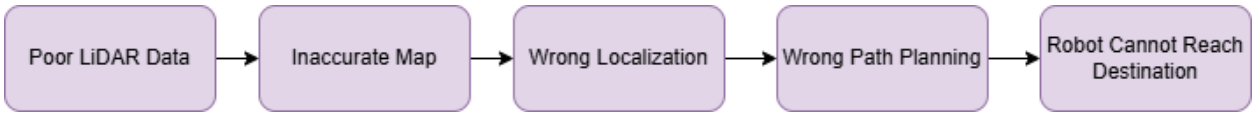


Figure 2.1: Problems Affecting Autonomous Navigation

Autonomous navigation is when the Bveeta Mini robot can navigate to a specific location without human intervention. However, there are several issues that may affect the system's performance. The first problem is that at the beginning of the operation, the robot does not know what the map of the environment looks like, so it cannot figure out a safe route. In addition, if the data obtained from the LiDAR sensor is incomplete or inaccurate, the resulting map will also be less accurate and may cause the robot to choose the wrong route. The robot must also have a good estimate of its position in the localization process. If this process is incorrect, the robot will move in the wrong direction or not reach the desired coordinates. Furthermore, when the robot does not detect obstacles in front, the robot may collide with surrounding objects and damage the robot or the surrounding environment. The route generated by the system must also be appropriate. Otherwise, the robot will take longer to reach the destination and will use more battery power.

In addition, communication between the computer and the robot via ROS also plays an important role. Without communication, navigation commands cannot be sent to the robot, and the autonomous navigation system cannot function as intended. If the robot is directed with its destination coordinates in a Python script, it may also move to the wrong destination, or reverse navigation may not be completed in the correct order. Although the robot is controlled via the keyboard, the robot is not capable of operating autonomously, and the user still needs to control the robot's movement. Any changes in the environment that occur after the map is generated can also change the robot's trajectory or stop its movement if there are some obstacles that can no longer be avoided. Finally, if the robot's battery runs out during the navigation process, then the robot may not be able to navigate to all the specified coordinates. Therefore, all these problems must be addressed to enable the robot to navigate autonomously with higher accuracy, safety and efficiency. For more clarity, the Problem Analysis and Consequences in Table 2.1 have been displayed.

Table 2.1: Summary of Problems and Their Consequences in the Autonomous Navigation System

Problem	**Consequence**
Robot has no map	Robot does not know safe route
LiDAR is inaccurate	Map becomes inaccurate
Incorrect localization	Robot moves to wrong location
Obstacles not detected	Robot may hit objects
Non-optimal route	Travel takes longer and battery drains quickly
ROS communication fails	Robot does not receive commands
Incorrect Python coordinates	Robot fails to reach destination
Relying on keyboard	Robot is not truly autonomous
Environment changes	Robot needs to change route or stop
Battery is low	Robot does not have time to complete navigation

Overall, autonomous navigation involves several interrelated processes such as mapping, localization, path planning, system communication and obstacle detection. Failure in any of these processes can cause the robot to be unable to move to the desired location accurately, safely and efficiently. Therefore, it is necessary to ensure that all system components are functioning properly to ensure that the robot can perform autonomous navigation.

3.0 AUTONOMOUS NAVIGATION METHOD

Autonomous navigation is a method that allows a robot to move automatically to a specified destination without the need for continuous user control. In this project, the Bveeta Mini robot uses several methods and software that are interconnected to allow the robot to build a map of the environment, determine its current position and move autonomously to three specified coordinates. The methods used consist of the Robot Operating System (ROS) as the main platform, LiDAR sensors to scan the environment, Hector SLAM to produce maps, Dynamic Window Approach (DWA) Local Planner to control the robot's movement and RViz which is used for the localization and destination determination process. The combination of all these methods allows the robot to perform autonomous navigation more accurately and systematically.

3.1 Robot Operating System (ROS)

Robot Operating System (ROS) is the main platform used in this project. ROS functions as a communication system that connects all robot components such as LiDAR sensors, SLAM systems and navigation planners. Through ROS, each component can share data and work together as a complete system. In this project, all processes such as generating maps, reloading maps and running autonomous navigation are implemented using packages provided by ROS, examples of commands used are *roscore*, *rosrun* and *roslaunch*. Therefore, ROS facilitates the development of robot systems and provides various functions that support autonomous navigation.

3.2 LiDAR Sensor

Light Detection and Ranging (LiDAR) is a sensor used to scan the area around the robot. This sensor measures the distance between the robot and objects using a laser beam and produces real-time environmental data. In this project, data from LiDAR is used to identify the position of walls, obstacles and empty spaces before being processed by the SLAM system. LiDAR was chosen because it is capable of producing more accurate and stable distance readings, thus helping to produce clearer maps for the navigation process.

3.3 Simultaneous Localization and Mapping (SLAM)

After data is obtained from the LiDAR sensor, the SLAM process is used to generate a map of the environment. Bveeta Mini provides two SLAM methods, namely Gmapping SLAM and Hector SLAM. However, in this project, Hector SLAM was chosen via the *roslaunch hectormapping bveeta_hector_slam.launch* command because this method is easy to use during experiments in the classroom to generate maps. Hector SLAM processes data from the LiDAR sensor to build a map of the area in real time while estimating the robot's position throughout the mapping process. After the map is generated, the map is saved and will be recalled using the

map_server, which is one of the ROS packages during the autonomous navigation process. This method allows the robot to use the same map without having to re-mapping again.

3.4 Dynamic Window Approach (DWA) Local Planner

After the map is successfully generated and reloaded, the robot needs a method to move to the selected destination. In this project, the navigation system is run using the roslaunch command *bveeta_navigation bveeta_dwa_local_planner.launch*, which shows that the Dynamic Window Approach (DWA) Local Planner is used as the navigation planner. It functions to control the robot's movement based on the loaded map and obstacle information received from the LiDAR sensor. With this method, the robot can generate an appropriate path, adjust the direction of movement when there are obstacles and move to the destination more safely.

3.5 Robot Visualization (RViz)

RViz is a visualization software used to display a map of the environment, the robot's position and the navigation process in real time. In this project, RViz is used after the map is recalled to ensure that the robot is in the correct position before navigation begins. The 2D Pose Estimate method is used to align the robot's actual position (red line on the map in rviz) with its position on the map, while the 2D Nav Goal method is used to set the destination that the robot wants to reach. After the destination is selected, the navigation system will automatically generate a route based on the loaded map and the robot will move to that location. The use of these two functions helps ensure that the robot can move more accurately and systematically during the autonomous navigation process.

4.0 AUTONOMOUS NAVIGATION STEP

This section describes the steps taken to implement autonomous navigation using the Bveeta Mini robot. The process starts from setting up communication between the computer and the robot, generating a map using Hector SLAM, recalling the saved map, performing autonomous navigation and ending with the robot automatically moving to the three specified coordinates shown in Figure 4.1. The Figure 4.1 below shows the overall navigation step flowchart.

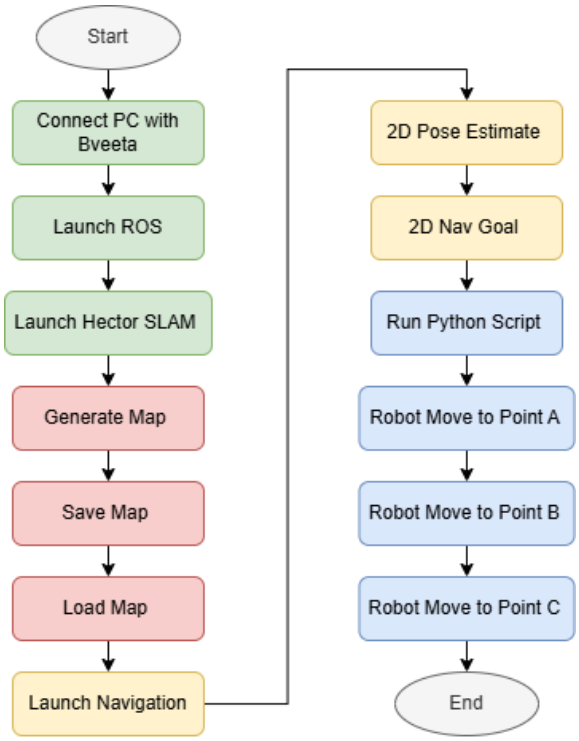


Figure 4.1: Overall Autonomous Navigation Step Flowchart.

Here for more detailed discussion from Figure 4.1 above, before the autonomous navigation process is carried out, communication between the computer and the Bveeta Mini robot needs to be established first. In this project, the connection is made using Remote Desktop Protocol (RDP) so that the computer can access the Ubuntu system on the robot. Before that, the Bveeta IP address has been obtained as shown in Figure 4.2 below to be entered into the Remote Desktop Connection. After the connection is successfully made, the roscore command is run as in Figure 4.3 to start the ROS Master which functions as the communication center for all ROS nodes throughout the autonomous navigation process.

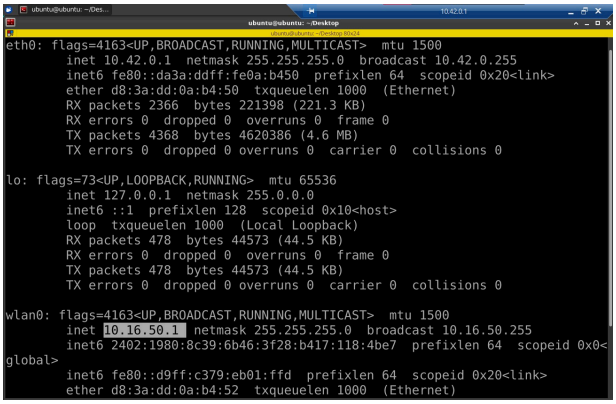


Figure 4.2: Bveeta IP Address from Ubuntu Desktop.

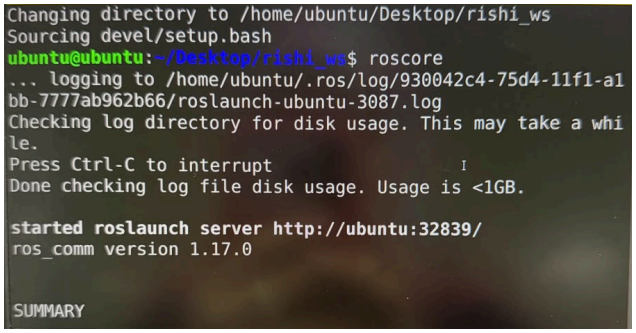


Figure 4.3: Roscore Command is Executed.

After communication is successfully established, the mapping process is started using Hector SLAM via the command *roslaunch hectormapping bveeta_hector_slam.launch*. The robot is then moved using the teleoperation keyboard as shown in Figure 4.4 below so that the LiDAR sensor can scan the entire area. The data from the LiDAR is processed by Hector SLAM to produce a real-time map of the environment that will be used during the autonomous navigation process. The resulting map of the environment that has been produced is shown in Figure 4.5.

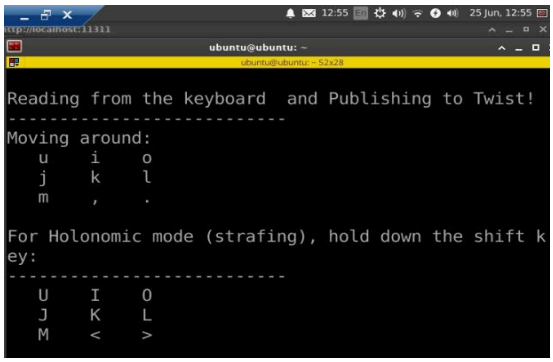


Figure 4.4: The Teleoperation Keyboard Command.

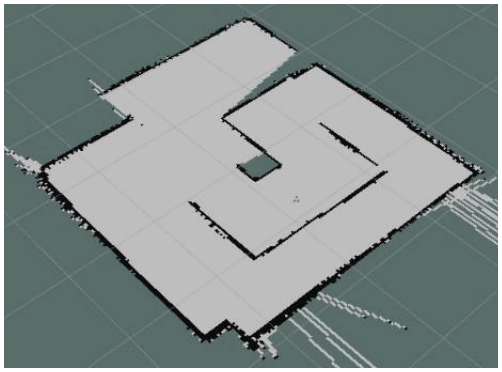


Figure 4.5: Map resulting from the corner class.

After the mapping process is complete, the generated map is saved using the *rosrun map_server map_saver -f Map1* command. The map is saved in the form of Map1.yaml and Map1.pgm files so that it can be reused without having to generate a new map each time navigation is performed. The saved map is recalled using the *rosrun map_server map_server Map1.yaml* command as shown in Figure 4.6 below. This step allows the navigation system to use the same map without having to rescan the surrounding area.

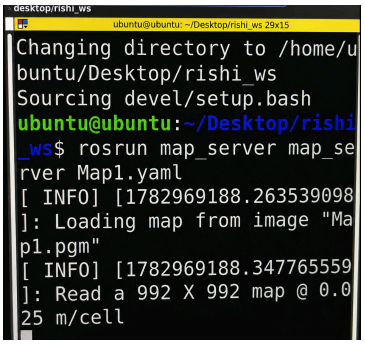


Figure 4.6: The command *rosrun map_server map_server Map1.yaml* has been used to call the saved map.

After the map is successfully loaded, the autonomous navigation system is run using the *roslaunch bveeta_navigation bveeta_dwa_local_planner.launch* command as shown in Figure 4.7. It is activated when the robot is positioned in the area that needs to be mapped. This command activates the navigation planner so that the robot can move based on the loaded map and avoid obstacles detected by the LiDAR sensor during the navigation process.

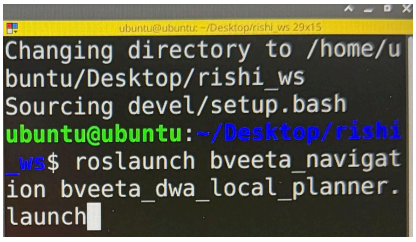


Figure 4.7: The command used after placing the robot on the map.

Before the robot moves to the destination, the 2D Pose Estimate function is used in RViz to align the robot's actual position with its position on the map as shown in Figure 4.8. This step is important to ensure that the robot knows its starting location accurately before navigation is carried out. After the localization process is complete, the 2D Nav Goal function is used to select the destination the robot wants to reach. When the destination is selected on the map, the navigation system will automatically generate a route and the robot starts moving towards that location. The 2D Pose Estimate and 2D Nav Goal functions can be found in RViz as shown in Figure 4.9.



Figure 4.8: The red line represents the actual position of the robot.

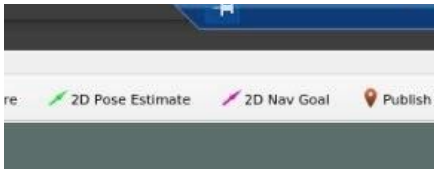


Figure 4.9: Two important functions are at the top of the RViz interface.

Before running sequential goal navigation, the Tutorial1_Sequential_goal.py file is first copied and saved using a new name, T1_anisdahlia.py, to avoid changes to the original file. Next, the terminal is opened and the file directory is verified using the '*ls'* command. The Python file is then converted to executable mode using the *chmod 777 T1_anisdahlia.py* command. The file name will change to green as shown in Figure 4.10. After that, ROS Master is run using the *roscore* command and the generated map is recalled as in the previous step.

To obtain the coordinates of each destination, the robot is placed at Point A and the *rostopic echo /clicked_point* commands are used as shown in Figure 4.11 to read the selected coordinates in RViz. Then, the location is selected using the Publish Point function and the coordinate values displayed on the terminal are recorded. The same process is repeated for all required destinations. The coordinate values are entered into the Python file as the position of each goal using the *gedit* command. After all the coordinates are updated, the program is run using the *python3 T1_anisdahlia.py* command. The robot then moves autonomously according to

the set coordinate sequence until it successfully reaches Point A, Point B and Point C as shown in Figure 4.12 without the need for keyboard control.

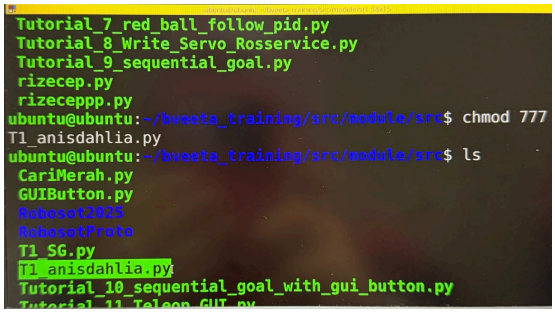


Figure 4.10: The file T1_anisdahlia.py shows it can be used after turning green.

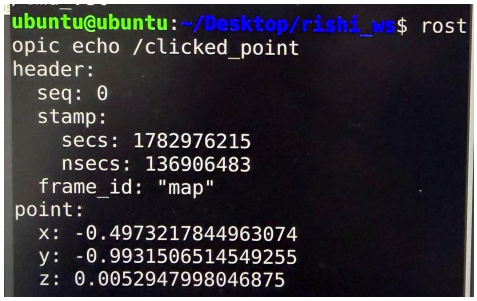


Figure 4.11: The command used to read the selected coordinates in RViz.

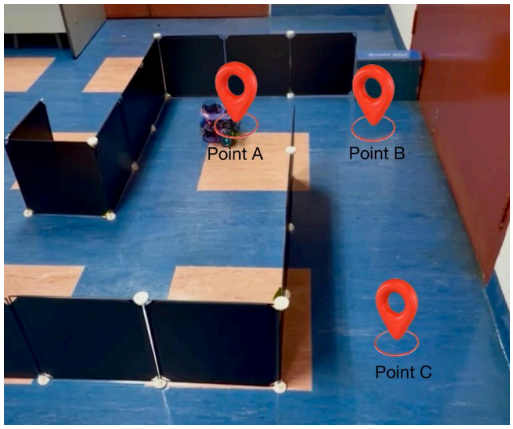


Figure 4.12: Point A, Point B and Point C that have been set coordinates in the python file.

5.0 EXPERIMENTAL RESULTS AND DISCUSSION

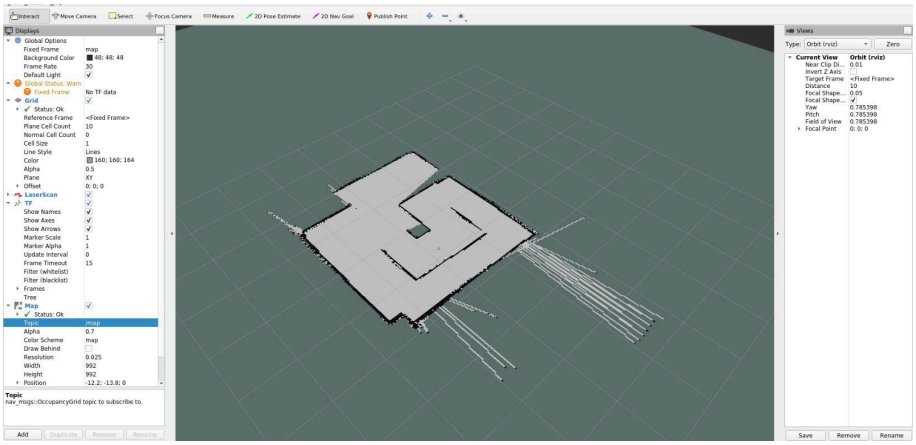


Figure 5.1: Generated map using Hector SLAM in RViz.

Figure 5.1 shows a map of the environment successfully generated using Hector SLAM based on data from the LiDAR sensor. This map will be used during the autonomous navigation process to enable the robot to identify areas and avoid obstacles.

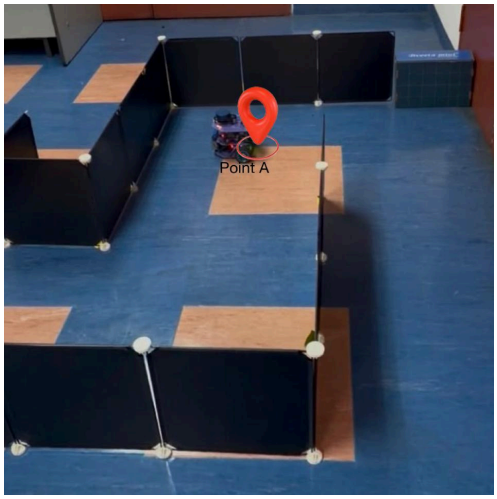


Figure 5.2: Robot successfully reached Point A.

Figure 5.2 shows the robot successfully moved and detected autonomously from the starting position to Point A based on the coordinates set in the Python script. The robot's movement was smooth and successfully detected the first destination.

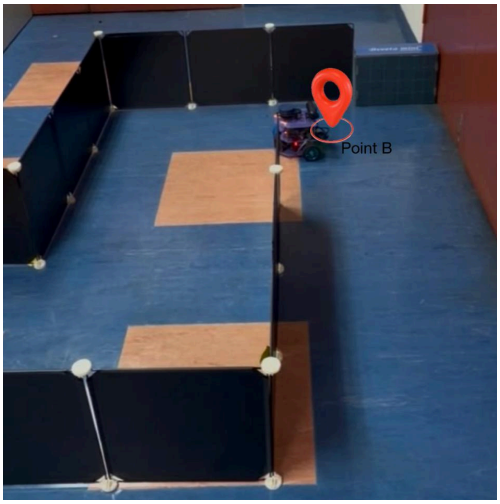


Figure 5.2: Robot successfully reached Point B.

Figure 5.2 shows after arriving at Point A, the robot automatically continues its journey to Point B without the need for keyboard control. The robot successfully navigates the route based on the loaded map.

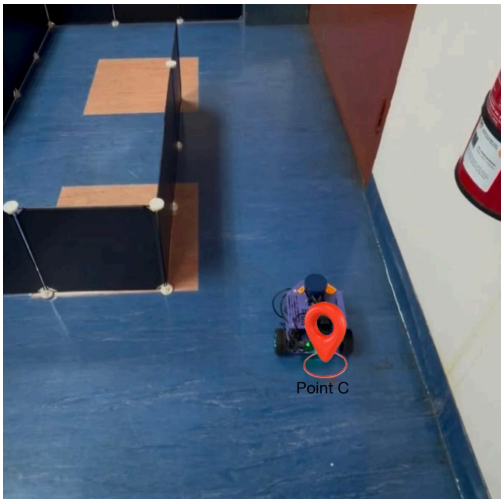


Figure 5.2: Robot successfully reached Point C.

Figure 5.6 shows the robot successfully completing the journey to Point C, which is the final destination. This shows that the autonomous navigation system successfully took the robot to all the specified coordinates.

6.0 SAFETY AND ENVIRONMENTAL CONSIDERATION

During implementation of the autonomous navigation system, one of the important things that needs to be taken into consideration is the safety and environmental consideration. When doing the testing, the Bveeta Mini robot was operated in a controlled indoor environment with predefined navigation area to minimize the risk of collisions with people, surrounding equipment or other robots. The robot localization and generated path were verified before running each navigation trial. This is to ensure that the planned trajectory was safe and free from unexpected obstacles. Furthermore, users kept a safe distance from the robot while it was moving autonomously, and an emergency stop procedure was established in case the robot behaved unexpectedly. By measuring the safety, it helped to reduce the risk of accidents and ensure that the experiment could be carried out in a controlled and safe manner.

From an environmental point of view, LiDAR based autonomous navigation minimizes unnecessary robot movement by generating efficient paths between destinations. This reduces energy use and increases the working time of the robot's battery. Moreover, the mapping process reuses previously generated maps instead of repeatedly scanning the same environment over again. This results in reducing computational workload and improving overall system efficiency. The rechargeable battery used by the robot also helps reduce disposable battery waste, contributing to more sustainable operation. Overall, the combination of adequate safety standards and an effective ROS-based autonomous navigation system demonstrate that the Bveeta Mini robot can operate safely, reliably and efficiently. These considerations promote the development of sustainable robotic applications that might be implemented in industrial, logistics and service sectors where safety, efficiency and environmental responsibility are equally vital.

7.0 CONCLUSION

To conclude, this project has successfully constructed an autonomous navigation system for the Bveeta Mini robot utilizing the Robot Operating System (ROS). Throughout this project, communication between the computer and the robot was successfully established and the robot was able to accomplish the required navigation tasks. The robot was able to construct a map of the surrounding environment, estimate its position accurately and move independently by using the LiDAR sensor together with the Hector SLAM algorithm and the ROS navigation stack with the Dynamic Window Approach (DWA) Local Planner. Based on the experimental results, the robot was able to run autonomously to the three predetermined destinations in the right direction by running the developed Python program. The robot was also able to avoid obstacles while following the planned path which proved that the mapping, localization and navigation systems were working successfully throughout the experiment. The results show that the navigation system is able to navigate autonomously in a reliable and accurate way under the testing conditions. To sum up, the objective of this project has been achieved. Other than showing the demonstration of the implementation of an autonomous navigation, this project also gave a better understanding of how different ROS components, LiDAR mapping and navigation algorithms operate together as a whole system. The safety procedures and the environmental considerations used in the experiment also contributed to the safe and efficient operation of the robot. The knowledge and experience gained from this project can also serve as a valuable foundation for the future development of more advanced autonomous robotic systems with improved navigation performance and wider real-world applications

8.0 REFERENCES

[1] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler and A. Y. Ng, "ROS: An Open-Source Robot Operating System," *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA) Workshop on Open Source Software*, Kobe, Japan, 2009.

[2] S. Kohlbrecher, O. von Stryk, J. Meyer and U. Klingauf, "A Flexible and Scalable SLAM System with Full 3D Motion Estimation," *IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, Kyoto, Japan, 2011.

[3] D. Fox, W. Burgard and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," *IEEE Robotics and Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.